

ГУАП

КАФЕДРА № 44

ПРЕПОДАВАТЕЛЬ

доцент, к.т.н., доцент

должность, уч. степень, звание

подпись, дата

А. М. Сергеев

инициалы, фамилия

ПРАКТИЧЕСКАЯ РАБОТА (ПРОЕКТ)

Обучение модели сортировки мусора

по курсу:

Основы искусственного интеллекта

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4341В

подпись, дата

Р.С. Кулаков

инициалы, фамилия

Санкт-Петербург

2026

1 Постановка задачи

1.1 Цель

Обучить модель определять тип мусора по фотографии, реализовать удобный интерфейс

1.2 Классы:

1. Картон
2. Стекло
3. Металл
4. Бумага
5. Пластик
6. Органика

Вход: фотография с мусором

Выход: предсказание класса + уверенность модели в %

2 Используемые технологии

2.1 Данные

Датасет Garbage Classification с Kaggle — ~2500 фото, уже разбитых по папкам

2.2 Модель

MobileNetV2 — лёгкая предобученная сеть, которую необходимо дообучить на данных

2.3 Обучение

Google Colab - бесплатный облачный сервис для программирования на Python

TensorFlow / Keras - открытая высокоуровневая библиотека глубокого обучения на языке Python

Интерфейс

Gradio — Python-библиотека с открытым исходным кодом, позволяющая создавать интерактивные веб-интерфейсы

1 Введение

Современный мир сталкивается с серьёзной экологической проблемой управления твёрдыми бытовыми отходами. По оценкам Всемирного банка, ежегодно в мире образуется более двух миллиардов тонн отходов, и эта цифра продолжает расти. При этом эффективность их переработки во многом зависит от качества предварительной сортировки: смешанный мусор практически невозможно переработать экономически целесообразным способом, так как разделение отходов на материалы вручную требует значительных трудозатрат и приводит к удорожанию конечной вторичной продукции.



Рисунок 1 – Свалка у дороги

Традиционный подход к сортировке отходов включает либо самостоятельное разделение мусора потребителями по категориям, либо ручную сортировку на специализированных предприятиях. Оба варианта имеют существенные недостатки: население часто допускает ошибки из-за недостатка знаний о составе материалов, а ручная сортировка медленна, дорога и физически тяжела для работников. По этой причине автоматизация классификации мусора с помощью технологий искусственного интеллекта становится одним из перспективных направлений развития отрасли переработки.

В последние годы технологии глубокого обучения открыли новые возможности для автоматизации задач компьютерного зрения. Современные алгоритмы способны с высокой точностью распознавать тип материала по фотографии, что позволяет создавать как промышленные системы сортировки на конвейерных линиях, так и пользовательские приложения, помогающие людям правильно сортировать мусор.

Особое значение в задачах классификации изображений имеют свёрточные нейронные сети (Convolutional Neural Networks, CNN). С появлением архитектур AlexNet (2012), VGG (2014), ResNet (2015) и более поздних — Inception, MobileNet, EfficientNet — точность распознавания на стандартных бенчмарках значительно превзошла человеческую. Развитие концепции transfer learning сделало эти технологии доступными широкому кругу разработчиков: больше нет необходимости в собственных вычислительных кластерах и миллионах размеченных изображений — достаточно адаптировать уже обученную модель к своей задаче.

Помимо точности, важным аспектом современных систем ИИ становится их объяснимость. Решения, выдаваемые ими, вызывают всё больше вопросов в чувствительных к ошибкам областях — медицине, праве, финансах. Сообщество исследователей развивает методы объяснимого искусственного интеллекта (Explainable AI, XAI), позволяющие визуализировать и интерпретировать решения нейросетевых моделей.

Целью настоящей работы является разработка системы автоматической классификации мусора по фотографии с использованием современных методов компьютерного зрения, а также реализация визуализации принятия решений моделью с помощью алгоритма Grad-CAM. В рамках проекта изучены теоретические основы свёрточных нейронных сетей и transfer learning, обучена собственная модель на открытом датасете, создан интерактивный веб-интерфейс с предзагруженными примерами и визуализацией внимания модели.

2 Обзор предметной области

2.1 Проблема классификации отходов

Бытовые отходы условно делятся на несколько крупных категорий, каждая из которых требует своего способа переработки. Бумага и картон представляют собой целлюлозосодержащие отходы, перерабатываемые в новую бумажную продукцию; стекло переплавляется и используется повторно; металлы — алюминий, сталь и другие — являются ценным вторичным сырьём; пластик представляет наиболее сложную категорию из-за множества типов полимеров с разными свойствами переработки; органические отходы компостируются; наконец, в категорию «прочий мусор» попадают отходы, не подлежащие переработке существующими методами.

Ключевая сложность задачи автоматической классификации заключается в высокой вариативности внешнего вида отходов в каждой категории. Бутылка может быть стеклянной, пластиковой или металлической; пакет — бумажным или полиэтиленовым; упаковка — комбинированной из нескольких материалов. Цвет, форма, степень загрязнения, освещение — всё это сильно влияет на сложность распознавания. По этой причине традиционные алгоритмы компьютерного зрения, основанные на ручном выделении признаков, не справляются с задачей; необходимы методы, способные автоматически обучаться различающим признакам по большому количеству примеров.

2.2 Существующие решения

В области автоматизированной сортировки мусора уже существуют коммерческие и исследовательские проекты. Компания AMP Robotics разрабатывает роботизированные системы сортировки на базе глубокого обучения; их установки работают на десятках предприятий по переработке в США и Европе. Финская компания ZenRobotics создаёт аналогичные решения для промышленного применения. Бренд Bin-E производит «умные» мусорные баки с автоматической сортировкой для офисов и общественных пространств. Существует множество

мобильных приложений, помогающих пользователям определить тип материала и найти ближайший пункт приёма.

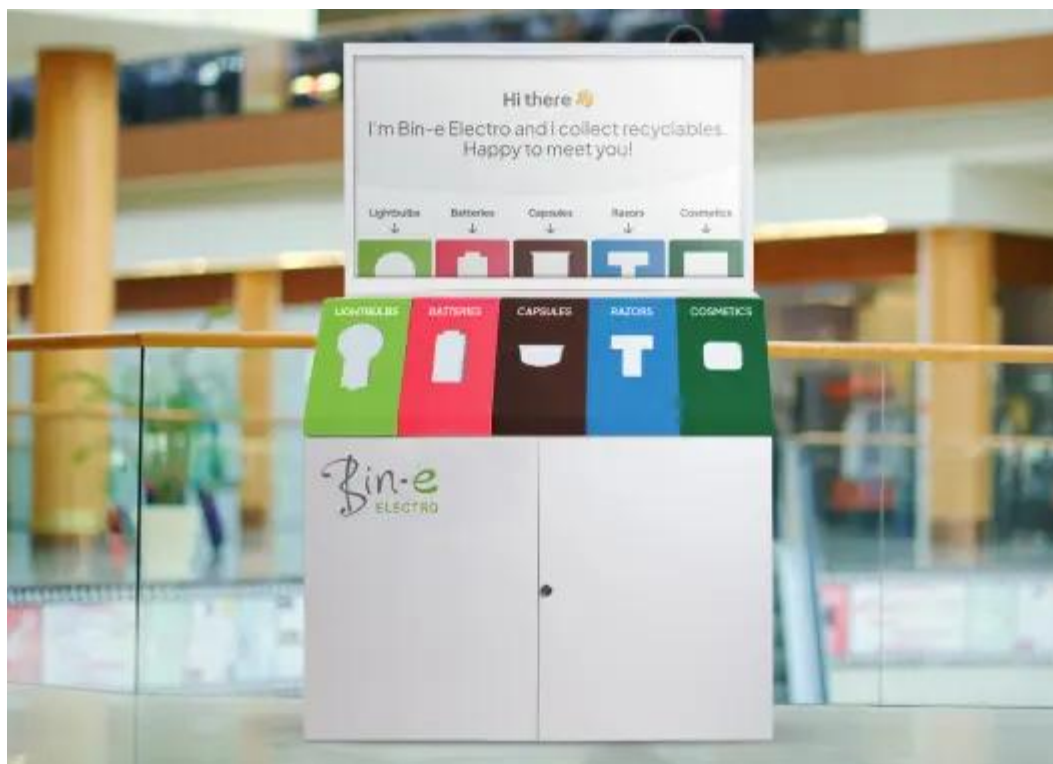


Рисунок 2 – «Умный» мусорный бак от Bin-E

В академической среде задача классификации мусора активно исследуется с использованием различных архитектур нейронных сетей. Открытый датасет Garbage Classification, использованный в данной работе, стал стандартным бенчмарком для подобных исследований и позволяет сравнивать результаты различных подходов.

3 Теоретические основы

3.1 Машинное обучение и нейронные сети

Машинное обучение — это раздел искусственного интеллекта, изучающий алгоритмы, способные обучаться на данных, то есть улучшать качество решения задач без явного программирования правил. В задачах классификации алгоритм получает на вход размеченный набор примеров, где каждому объекту сопоставлен правильный класс, и на основе этих данных строит модель, способную предсказывать класс для новых, ранее не виденных объектов.

Нейронные сети — один из подходов в машинном обучении, изначально вдохновлённый строением биологических нейронных систем, но в современном виде представляющий собой просто параметрический математический аппарат. Базовая единица сети — искусственный нейрон, выполняющий простую операцию: получает несколько входных значений, умножает каждое на свой вес, суммирует результаты, прибавляет смещение и пропускает итог через функцию активации. Нейроны организованы в слои, выход одного слоя подаётся на вход следующего.

Обучение сети заключается в подборе значений весов и смещений таким образом, чтобы минимизировать разность между предсказаниями сети и правильными ответами на обучающей выборке. Эта разность измеряется функцией потерь (loss function); для задач многоклассовой классификации используется категориальная кросс-энтропия. Процесс оптимизации обычно осуществляется методом обратного распространения ошибки в сочетании с градиентным спуском или его модификациями

В данной работе использовался оптимизатор Adam — адаптивный метод, который автоматически подстраивает скорость обучения для каждого параметра.

3.2 Свёрточные нейронные сети

Обычные полносвязные нейронные сети плохо подходят для работы с изображениями: каждый пиксель становится отдельным признаком, и количество

параметров стремительно растёт с разрешением картинки. Кроме того, такая сеть не учитывает пространственную структуру изображения — соседние пиксели обрабатываются так же, как удалённые друг от друга. Свёрточные нейронные сети решают эти проблемы за счёт нескольких ключевых операций.

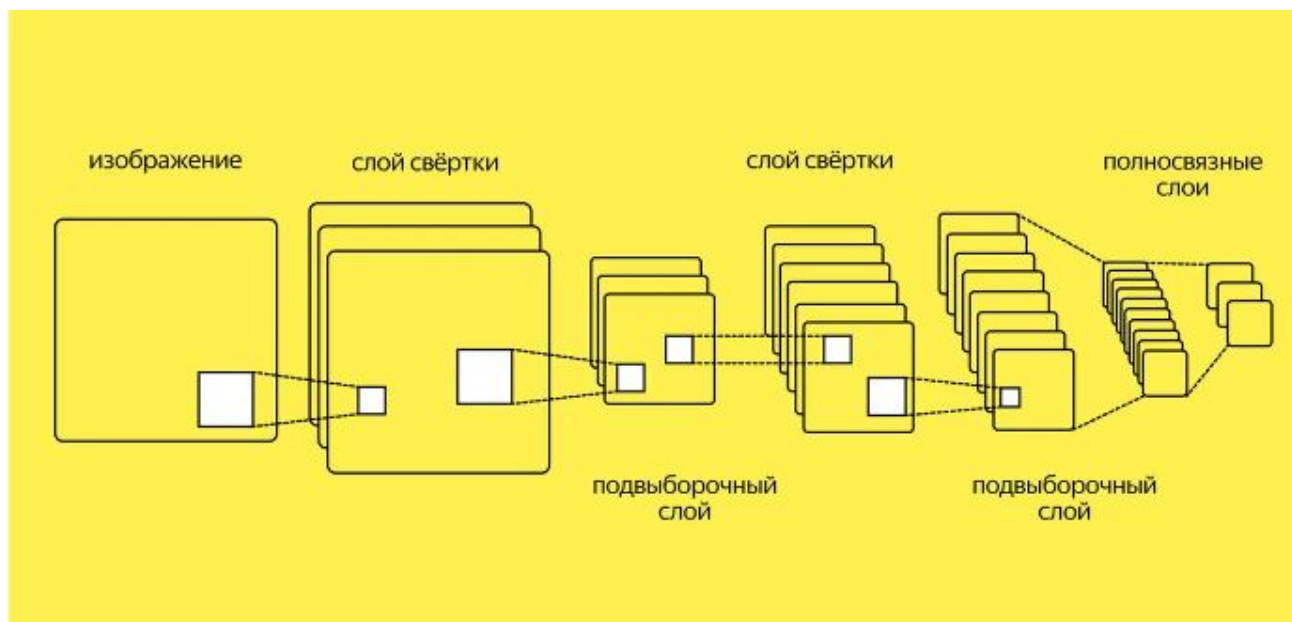


Рисунок 3 – Свёрточные нейронные сети

3.2.1 Свёртка (convolution)

Операция, при которой к исходному изображению применяется набор небольших фильтров (обычно 3×3 или 5×5 пикселей). Каждый фильтр сканирует изображение, выделяя определённый локальный паттерн — край, угол, элемент текстуры. Веса фильтров обучаются автоматически в процессе тренировки. Один свёрточный слой содержит десятки или сотни фильтров, выдающих столько же карт признаков. Принципиально, что один и тот же фильтр применяется ко всем участкам изображения — это даёт сети свойство трансляционной инвариантности и значительно сокращает количество обучаемых параметров.

3.2.2 Пулинг (pooling)

Операция пулинга уменьшает пространственный размер карт признаков, обычно беря максимум (max pooling) или среднее (average pooling) в небольших

окнах. Это снижает вычислительную нагрузку на последующих слоях и делает сеть более устойчивой к небольшим смещениям объекта.

3.2.3 Функция активации

Обеспечивает нелинейность преобразований, без которой многослойная сеть была бы эквивалентна одному линейному слою. Стандартный выбор для скрытых слоёв — ReLU (Rectified Linear Unit), возвращающая аргумент, если он положителен, и ноль в противном случае. Для выходного слоя в задачах многоклассовой классификации используется softmax — функция, преобразующая значения нейронов в вероятностное распределение по классам.

Типичная архитектура CNN представляет собой последовательность чередующихся свёрточных и пулинговых слоёв, постепенно уменьшающих пространственный размер карт признаков и увеличивающих их количество. На выходе обычно используется один или несколько полносвязных слоёв, завершающихся softmax-слоем размерности, равной числу классов.

3.3 Transfer Learning

Глубокие свёрточные сети современного уровня содержат от нескольких до сотен миллионов параметров. Их полноценное обучение с нуля требует огромных размеченных датасетов (миллионы изображений) и значительных вычислительных ресурсов — десятки и сотни часов работы на специализированном железе. Для большинства практических задач такие условия недостижимы.

Решением является transfer learning — методика, при которой модель, предварительно обученная на большом универсальном датасете, адаптируется к решению конкретной задачи. Идея основана на наблюдении, что нижние слои свёрточных сетей выделяют универсальные низкоуровневые признаки (края, цветовые градиенты, простые текстуры), применимые практически к любым изображениям. Более глубокие слои выделяют признаки среднего и высокого уровня

(формы, фрагменты объектов), которые становятся всё более специфичными для решаемой задачи.

В простейшем варианте transfer learning веса всех слоёв предобученной модели замораживаются — то есть не меняются в процессе обучения — а к ней добавляется новая классификационная «голова» в виде нескольких полносвязных слоёв, обучаемых на целевом датасете. При наличии достаточного количества данных может применяться fine-tuning — разморозка верхних слоёв базовой модели и их дообучение с малой скоростью обучения, что позволяет модели лучше адаптироваться к специфике задачи.

Стандартным «донором» весов в transfer learning является датасет ImageNet, содержащий более 14 миллионов изображений, размеченных по 1000 классам объектов реального мира. Модели, предобученные на ImageNet, оказались универсально применимыми к самым разным задачам компьютерного зрения — от медицинской диагностики до распознавания дорожной обстановки.

3.4 Архитектура MobileNetV2

Среди множества современных архитектур CNN сеть MobileNetV2, представленная исследователями Google в 2018 году, выделяется балансом между точностью и вычислительной эффективностью. Архитектура изначально создавалась для применения на мобильных устройствах и встраиваемых системах, где ресурсы ограничены.

Первая ключевая инновация архитектуры — depthwise separable convolutions, разделение операции свёртки на два более простых шага. Сначала к каждому каналу входной карты признаков отдельно применяется свёртка (depthwise convolution), затем выполняется свёртка 1×1 для объединения каналов (pointwise convolution). Это значительно сокращает число параметров и операций по сравнению с обычной свёрткой при сохранении выразительной способности.

Вторая инновация — *inverted residual blocks* с *linear bottlenecks*. Это особый тип остаточных соединений. В отличие от классических ResNet-блоков, где остаточные соединения связывают слои с большим числом каналов, в MobileNetV2 они связывают «узкие места» — слои с малым числом каналов, между которыми происходит временное расширение в более широкое пространство признаков. Также удалена нелинейность на выходе блока, что предотвращает потерю информации при сжатии размерности.

В результате MobileNetV2 содержит около 3,5 миллионов параметров и весит порядка 14 МБ, что в десять и более раз меньше популярных архитектур ResNet50 или VGG16. При этом точность классификации на ImageNet остаётся конкурентоспособной — около 72 % top-1 ассигура.

3.5 Регуляризация и борьба с переобучением

Одна из главных проблем при обучении нейронных сетей — переобучение. Это явление, при котором модель слишком хорошо запоминает обучающие примеры, теряя способность обобщать на новые данные. Признак переобучения — большой разрыв между точностью на обучающей и валидационной выборках: модель отлично работает на тех данных, что видела, но плохо — на новых.

В данной работе применялось несколько техник борьбы с переобучением. Аугментация данных представляет собой искусственное расширение обучающей выборки путём случайных преобразований исходных изображений: поворотов, отражений, масштабирования, сдвигов яркости. Каждый раз модель видит немного отличающиеся варианты тех же объектов, что заставляет её обучаться обобщённым признакам, а не запоминать конкретные детали отдельных фотографий.

Dropout — техника случайного выключения части нейронов на каждой итерации обучения. Это предотвращает чрезмерную специализацию отдельных нейронов и заставляет сеть распределять знания по большему числу путей. Во время инференса (применения обученной модели) dropout не применяется. В данной работе

использовался dropout с коэффициентом 0,3 — то есть на каждой итерации 30% нейронов соответствующего слоя случайно обнулялись.

Early stopping — техника, при которой обучение останавливается, когда метрика на валидационной выборке перестаёт улучшаться. Это предотвращает доучивание модели до состояния переобучения, а также экономит вычислительное время.

Замораживание весов базовой модели само по себе является формой регуляризации: поскольку дообучаются только верхние слои с относительно небольшим числом параметров, риск переобучения значительно снижается.

3.6 Объяснимый искусственный интеллект и Grad-CAM

С ростом сложности моделей машинного обучения возрастает и потребность в понимании их решений. Современные глубокие нейронные сети с миллионами параметров фактически являются «чёрными ящиками»: они выдают предсказания, но почему именно так — неочевидно. В критических областях, таких как медицинская диагностика, правовые системы или автономный транспорт, такая непрозрачность недопустима. Это породило целое направление исследований, известное как объяснимый ИИ (Explainable AI, XAI), занимающееся разработкой методов интерпретации решений сложных моделей.

Для задач компьютерного зрения одним из наиболее популярных и эффективных методов объяснения является Grad-CAM (Gradient-weighted Class Activation Mapping), предложенный в 2017 году. Метод позволяет визуализировать, какие участки входного изображения внесли наибольший вклад в принятие моделью конкретного решения о классе.

Принцип работы алгоритма следующий. Сначала изображение пропускается через обученную сеть, фиксируется итоговое предсказание и активации последнего свёрточного слоя — двумерных карт признаков, каждая из которых соответствует одному фильтру. Затем вычисляются градиенты предсказанного класса по этим картам признаков: мы оцениваем, как небольшое изменение значений в каждой карте повлияло бы на финальную оценку класса. Эти градиенты усредняются по пространственным измерениям, давая для каждой карты признаков единственное число — её важность для текущего предсказания. Карты признаков взвешиваются этими коэффициентами важности и суммируются в одну итоговую тепловую карту. К ней применяется функция ReLU — берутся только положительные значения. Полученная карта приводится к размеру исходного изображения через интерполяцию и накладывается на него с цветовым кодированием, обычно через цветовую палитру «jet»: синий цвет соответствует низкой важности участка, красный — высокой.

Преимущество Grad-CAM заключается в его универсальности: метод применим практически к любой CNN-архитектуре без необходимости её модификации и не требует дообучения. Полученные тепловые карты можно интерпретировать как «внимание» модели — области, на которые она смотрит при принятии решения. Это позволяет, во-первых, повышать доверие к модели в случаях, когда визуализация подтверждает разумность её решений, и, во-вторых, обнаруживать ошибки — например, ситуации, когда модель ориентируется на нерелевантные элементы фона или артефакты освещения.

4 Используемые технологии

4.1 Среда разработки Google Colab

Google Colab (Google Colaboratory) — облачный сервис от Google, предоставляющий бесплатный доступ к интерактивной среде Jupyter Notebook прямо в браузере. Главными преимуществами Colab являются отсутствие необходимости установки программного обеспечения локально, предустановленные библиотеки машинного обучения, а также бесплатный доступ к графическим процессорам (GPU) и тензорным процессорам (TPU). В рамках данной работы использовался GPU NVIDIA Tesla T4, который значительно ускорил процесс обучения нейронной сети — то, что на обычном процессоре заняло бы несколько часов, на GPU выполняется за несколько минут.



Рисунок 4 – Google Colaboratory

Среда Colab позволяет легко работать с внешними источниками данных, в частности с платформой Kaggle, и публиковать результаты работы через временные публичные ссылки. Также Colab обеспечивает встроенные средства визуализации:

графики, изображения и интерактивные виджеты могут отображаться прямо в ноутбуке.

4.2 Язык программирования Python

Python — высокоуровневый язык программирования, ставший де-факто стандартом в области искусственного интеллекта и анализа данных. Его выбор обусловлен богатой экосистемой библиотек для машинного обучения, простотой синтаксиса и широкой поддержкой сообщества разработчиков. На Python написаны практически все ключевые библиотеки в области ИИ: TensorFlow, PyTorch, Keras, scikit-learn, NumPy, Pandas и многие другие. В работе использовался Python версии 3.

4.3 Библиотеки TensorFlow и Keras

TensorFlow — открытая платформа для машинного обучения, разработанная Google и являющаяся одним из лидеров отрасли. Она предоставляет полный набор инструментов для создания, обучения и развёртывания моделей нейронных сетей, поддерживает работу как на CPU, так и на GPU/TPU, имеет средства для оптимизации моделей под мобильные устройства (TensorFlow Lite) и серверы (TensorFlow Serving).

В рамках TensorFlow используется высокоуровневый API Keras, который значительно упрощает построение моделей: вместо написания низкоуровневого кода для каждой математической операции разработчик описывает архитектуру сети в виде последовательности слоёв. В работе применялись следующие компоненты TensorFlow и Keras:

- модуль `tf.keras.applications` для загрузки предобученной модели MobileNetV2;
- класс `ImageDataGenerator` для подготовки и аугментации изображений;
- классы слоёв `Dense`, `Dropout`, `GlobalAveragePooling2D` для построения собственного классификатора;

- механизм `EarlyStopping` для предотвращения переобучения и автоматической остановки обучения;
- класс `GradientTape` для вычисления градиентов в реализации Grad-CAM.

4.4 Вспомогательные библиотеки

`NumPy` — фундаментальная библиотека для научных вычислений в Python, обеспечивающая эффективную работу с многомерными массивами. Использовалась для манипуляций с данными изображений и тепловыми картами.

`Matplotlib` — библиотека для построения графиков и визуализации данных. Применялась для отображения кривых обучения модели — графиков точности и функции потерь по эпохам.

`OpenCV (cv2)` — мощная библиотека компьютерного зрения, поддерживающая широкий спектр операций над изображениями. В данной работе она использовалась для применения цветовой палитры к тепловой карте Grad-CAM, изменения размера тепловой карты и её наложения на исходное изображение.

`Gradio` — библиотека Python для создания веб-интерфейсов к моделям машинного обучения буквально в несколько строк кода. Автоматически генерирует HTML-страницу с компонентами загрузки данных, запуска модели и отображения результатов. Также предоставляет возможность опубликовать интерфейс по временной публичной ссылке, что делает её удобным инструментом для демонстрации проектов.

`Kaggle CLI` — командный интерфейс платформы Kaggle, позволяющий автоматизированно скачивать датасеты с использованием персонального API-токена.

5 Описание датасета

В качестве источника данных был выбран открытый датасет Garbage Classification с платформы Kaggle. Датасет содержит 2527 изображений мусора, разделённых на шесть классов:

- Картон (cardboard) — 403 изображения;
- Стекло (glass) — 501 изображение;
- Металл (metal) — 410 изображений;
- Бумага (paper) — 594 изображения;
- Пластик (plastic) — 482 изображения;
- Прочий мусор (trash) — 137 изображений.

Все изображения представляют собой фотографии отдельных предметов на относительно однородном фоне, что упрощает задачу классификации. Размеры изображений варьируются, поэтому при загрузке они приводятся к единому разрешению 224×224 пикселя — стандартному входному размеру для MobileNetV2.

Можно заметить значительный дисбаланс классов: категория «прочий мусор» представлена существенно меньшим количеством изображений по сравнению с другими (137 против 400–600 у остальных). Это потенциально может снизить точность классификации этого класса, что важно учитывать при анализе результатов. В рамках данной работы дисбаланс не корректировался дополнительными методами (например, перевзвешиванием классов или генерацией синтетических примеров), однако в качестве направления развития проекта эти подходы могут быть применены.

Для борьбы с переобучением и увеличения разнообразия обучающих данных была применена аугментация — техника искусственного расширения датасета с помощью случайных преобразований исходных изображений. В работе использованы следующие виды аугментации: случайный поворот на угол до 20 градусов; горизонтальное отражение; случайное масштабирование (zoom) в пределах 20 %. Аугментация применяется только к обучающей выборке;

валидационная выборка остаётся неизменной для корректной оценки качества. Датасет был разделён на обучающую (80 %) и валидационную (20 %) выборки случайным образом.

6 Реализация классификатора

6.1 Подготовка среды

Работа выполнялась в Google Colab с включённым GPU-ускорителем. Все необходимые библиотеки (TensorFlow, Keras, NumPy, Matplotlib, OpenCV) предустановлены в Colab по умолчанию. Дополнительно были установлены библиотеки Gradio и Kaggle CLI:

```
!pip install gradio kaggle -q
```

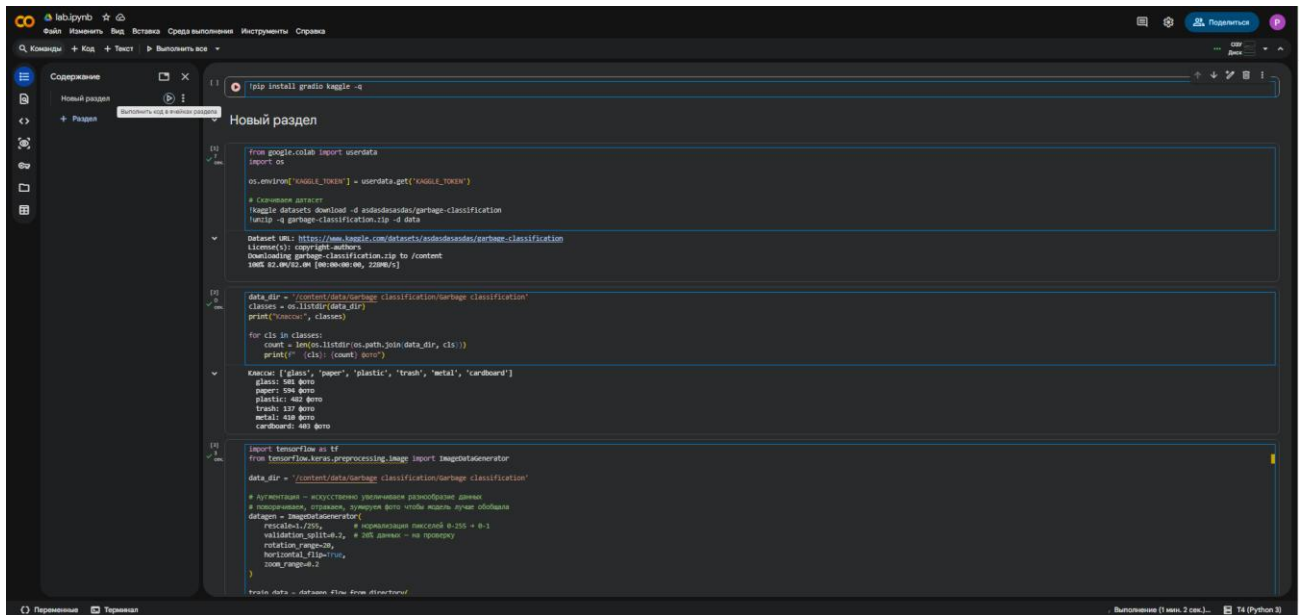


Рисунок 5 – Интерфейс Google CoLab

6.2 Авторизация и загрузка датасета

Для автоматизированной загрузки датасета был использован API Kaggle. После регистрации на платформе пользователь генерирует персональный API-токен в настройках своего аккаунта. Этот токен передаётся в переменные окружения, после чего датасет скачивается напрямую в Colab без необходимости ручной загрузки:

```
import os
os.environ['KAGGLE_TOKEN'] = '<токен>'
!kaggle datasets download -d asdasdasdas/garbage-classification
!unzip -q garbage-classification.zip -d data
```

6.3 Подготовка данных

Для подготовки изображений использовался класс `ImageDataGenerator` из Keras. Он автоматически читает файлы из директорий, при этом каждая поддиректория интерпретируется как отдельный класс. Класс также выполняет нормализацию пикселей (приведение значений из диапазона 0–255 к диапазону 0–1), что улучшает сходимость обучения, и применяет аугментацию:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    horizontal_flip=True,
    zoom_range=0.2
)

train_data = datagen.flow_from_directory(
    data_dir, target_size=(224, 224),
    batch_size=32, subset='training'
)
val_data = datagen.flow_from_directory(
    data_dir, target_size=(224, 224),
    batch_size=32, subset='validation'
)
```

Параметр `batch_size=32` определяет, что модель будет обучаться партиями по 32 изображения за один шаг — это компромисс между скоростью обучения и стабильностью градиентов.

6.4 Построение архитектуры модели

Архитектура классификатора построена по принципу `transfer learning`. В качестве базовой модели использована `MobileNetV2` с предобученными на ImageNet весами, при этом её выходной классификационный слой удалён (параметр `include_top=False`). Все слои базовой модели заморожены — их веса не обновляются в процессе обучения.

Поверх базовой модели добавлены собственные слои классификатора. Слой `GlobalAveragePooling2D` преобразует двумерные карты признаков размером $7 \times 7 \times 1280$ в одномерный вектор длиной 1280, усредняя значения по пространственным измерениям. Полносвязный слой из 128 нейронов с функцией

активации ReLU обучается отображать признаки в промежуточное представление, адаптированное под задачу классификации мусора. Слой Dropout с коэффициентом 0,3 случайным образом обнуляет 30 % активаций для борьбы с переобучением. Наконец, выходной полносвязный слой из 6 нейронов с функцией активации softmax выдаёт распределение вероятностей по классам:

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models

base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(6, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Общее количество параметров модели составляет 2 422 726, из которых обучаемыми являются только 164 742 (около 6,8 %) — это параметры добавленного классификатора. Оставшиеся 2 257 984 параметра заморожены, что и обеспечивает эффект transfer learning. Модель скомпилирована с оптимизатором Adam и функцией потерь categorical_crossentropy — стандартный выбор для задач многоклассовой классификации.

6.5 Обучение модели

Обучение проводилось на протяжении до 10 эпох с применением механизма EarlyStopping, автоматически останавливающего обучение, если в течение 3 эпох не

наблюдается улучшения точности на валидационной выборке. Параметр `restore_best_weights=True` гарантирует, что по завершении обучения модель будет содержать веса с наилучшим показателем валидационной точности, а не последние:

```
history = model.fit(
    train_data,
    epochs=10,
    validation_data=val_data,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            patience=3,
            restore_best_weights=True
        )
    ]
)
```

Каждая эпоха обучения занимала примерно 30–60 секунд на GPU T4, что в сумме дало время обучения порядка 5–10 минут. В процессе обучения модель прошла обучающую выборку несколько раз, на каждой итерации обновляя веса в направлении уменьшения функции потерь.

6.6 Анализ кривых обучения

После завершения обучения построены графики метрик по эпохам — отдельно для обучающей и валидационной выборок. Это позволяет визуально оценить динамику обучения:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Обучение')
plt.plot(history.history['val_accuracy'], label='Валидация')
plt.title('Точность'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Обучение')
plt.plot(history.history['val_loss'], label='Валидация')
plt.title('Ошибка'); plt.legend()
plt.show()
```

Кривые обучения показали стабильный рост точности и снижение функции потерь, что свидетельствует о корректной настройке гиперпараметров. Расхождение между обучающей и валидационной кривыми остаётся в разумных пределах, что говорит об отсутствии явного переобучения.

7 Реализация визуализации внимания через Grad-CAM

7.1 Концептуальная схема

Реализация Grad-CAM представляет собой нетривиальную инженерную задачу, поскольку требует работы с внутренними слоями нейросетевой модели, имеющей сложную (вложенную) структуру. В нашем случае модель имеет структуру Sequential, первым элементом которой является функциональная модель MobileNetV2, а остальными — слои собственного классификатора.

Чтобы получить градиенты предсказанного класса по картам признаков последнего свёрточного слоя MobileNetV2, необходимо построить вспомогательную модель с двумя выходами: активациями интересующего нас свёрточного слоя и выходом базовой модели. Затем нужно вручную провести данные через оставшиеся слои классификатора внутри блока GradientTape, чтобы автоматическое дифференцирование TensorFlow зафиксировало граф вычислений.

Разберём ключевые шаги. Сначала из основной модели извлекается базовая часть — MobileNetV2, и в ней находится последний свёрточный слой по имени Conv_1. Затем строится вспомогательная модель feature_model, имеющая один вход и два выхода: активации Conv_1 и итоговый выход базовой модели. Внутри блока GradientTape, который записывает все вычисления для последующего вычисления градиентов, входное изображение пропускается через эту модель, а полученный выход вручную проводится через оставшиеся слои классификатора. После получения предсказаний определяется индекс класса с наибольшей вероятностью.

Далее вычисляются градиенты класса с наибольшей вероятностью по карте признаков Conv_1. Эти градиенты усредняются по пространственным измерениям, давая вектор важности каждого канала карты признаков. Карта признаков умножается на этот вектор (поканально), и результат суммируется по каналам, давая итоговую двумерную тепловую карту. К ней применяется операция ReLU (через tf.maximum), и она нормируется в диапазон от 0 до 1 для последующего отображения.

7.2 Наложение тепловой карты на изображение

Полученная тепловая карта имеет размер 7×7 пикселей (соответствующий пространственной размерности последнего свёрточного слоя MobileNetV2 при входе 224×224). Для отображения её необходимо увеличить до размера исходного изображения, окрасить в цветовую палитру и наложить на оригинал:

```
def overlay_heatmap(img, heatmap, alpha=0.4):
    heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap_colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)
    overlay = cv2.addWeighted(
        img.astype(np.uint8), 1 - alpha,
        heatmap_colored, alpha, 0
    )
    return overlay
```

Здесь карта приводится к размеру изображения с помощью билинейной интерполяции (`cv2.resize`), переводится в формат 8-битного беззнакового целого, окрашивается с помощью палитры «jet» (от синего через зелёный и жёлтый к красному), приводится из BGR в RGB (так как OpenCV использует BGR, а другие библиотеки — RGB) и накладывается на исходное изображение с прозрачностью 60%/40 %. Параметр `alpha` регулирует интенсивность наложения: при `alpha=0` видно только оригинал, при `alpha=1` — только цветную карту.

8 Веб-интерфейс на Gradio

8.1 Архитектура интерфейса

Для удобного тестирования модели на собственных изображениях был создан интерактивный веб-интерфейс с помощью библиотеки Gradio. Интерфейс состоит из одного входа (поле загрузки изображения) и двух выходов: метки с вероятностями классов и изображения с наложенной тепловой картой Grad-CAM. Дополнительно под полем загрузки отображается галерея готовых примеров — пользователь может кликнуть по любому из них и сразу увидеть, как работает модель.

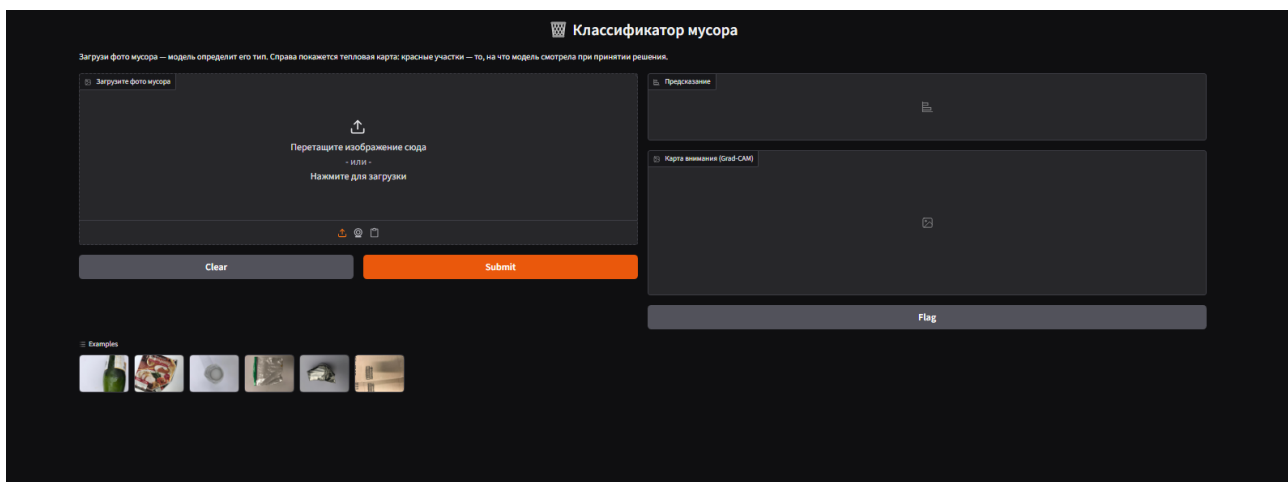


Рисунок 6 – Интерфейс разработанного сайта

8.2 Функция предсказания

Основная функция объединяет всё описанное выше — классификацию и Grad-CAM. Изображение приводится к нужному размеру и нормируется. Затем выполняется предсказание модели, формируется словарь «класс → вероятность» для удобного отображения, рассчитывается тепловая карта Grad-CAM и накладывается на изображение. Функция возвращает оба результата, которые Gradio автоматически отобразит в соответствующих компонентах интерфейса.

8.3 Готовые примеры

Для повышения удобства взаимодействия в интерфейс добавлены примеры — по одному случайно выбранному изображению из каждого класса. Это позволяет

пользователю быстро протестировать систему, не подыскивая собственные фотографии:

```
example_paths = []
for cls in os.listdir(data_dir):
    cls_dir = os.path.join(data_dir, cls)
    files = os.listdir(cls_dir)
    example_paths.append(os.path.join(cls_dir, random.choice(files)))
```

8.4 Создание и публикация интерфейса

Сам интерфейс создаётся с помощью класса `gr.Interface`:

```
app = gr.Interface(
    fn=predict,
    inputs=gr.Image(label="Загрузите фото мусора"),
    outputs=[
        gr.Label(num_top_classes=6, label="Предсказание"),
        gr.Image(label="Карта внимания (Grad-CAM)")
    ],
    title="Классификатор мусора",
    description=(
        "Загрузите фото мусора – модель определит его тип. "
        "Справа покажется тепловая карта: красные участки – "
        "то, на что модель смотрела при принятии решения."
    ),
    examples=example_paths
)

app.launch(share=True)
```

Параметр `share=True` создаёт временную публичную ссылку, по которой интерфейс доступен с любого устройства в Интернете. Это позволяет демонстрировать работу системы без необходимости настраивать собственный сервер.

9 Результаты и анализ

9.1 Количественные результаты

В результате обучения была получена модель, достигшая высокой точности классификации на валидационной выборке. График обучения показывает стабильный рост точности и снижение функции потерь, что свидетельствует о корректной настройке гиперпараметров и отсутствии явного переобучения. Точность на обучающей и валидационной выборках сходится к близким значениям, что говорит о хорошей способности модели к обобщению.

Применение transfer learning позволило достичь высоких показателей за минимальное время обучения. Если бы модель обучалась с нуля на доступном объёме данных (около 2500 изображений), точность была бы значительно ниже — для глубоких нейронных сетей обычно требуются десятки и сотни тысяч изображений на класс. Использование предобученных весов фактически позволило перенести в модель знания, накопленные при обучении на миллионах изображений ImageNet.

9.2 Качественный анализ через Grad-CAM

Реализация визуализации Grad-CAM позволила провести качественный анализ работы модели. Тепловые карты на корректно классифицированных изображениях демонстрируют, что модель смотрит на сам объект, а не на фон или артефакты освещения. Например, при классификации стеклянной бутылки активность сосредоточена в области бутылки и её бликов; при классификации картонной коробки — на текстуре и рёбрах картона.

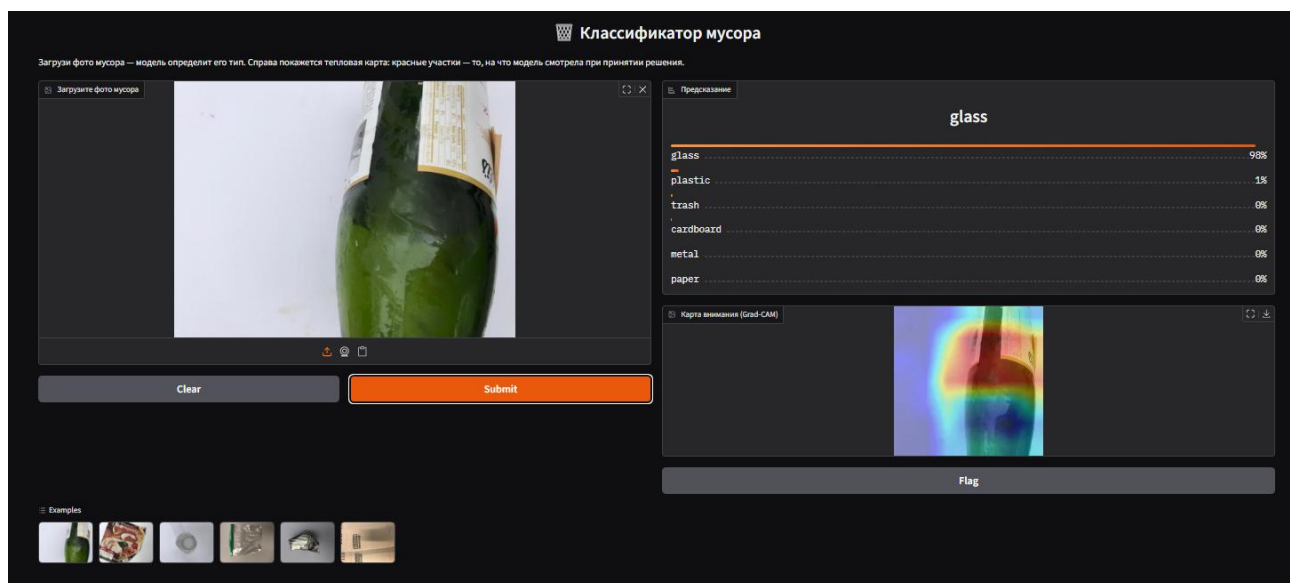


Рисунок 7 – Пример результата классификации (стекло)

Анализ ошибок через Grad-CAM также информативен. В случаях неправильной классификации тепловая карта часто показывает, что модель ориентировалась на нерелевантные элементы — фон, тени, посторонние объекты. Это даёт ценные подсказки для дальнейшего улучшения системы: можно фокусироваться на сборе более разнообразных обучающих данных, на удалении фона из тренировочных изображений или на применении более сильной аугментации.

Качественное тестирование показало, что модель уверенно распознаёт большинство видов мусора, особенно те классы, в которых было больше обучающих изображений (бумага, стекло, пластик). Класс «прочий мусор» (trash) показал ожидаемо более низкую точность из-за меньшего количества обучающих примеров и большего внутреннего разнообразия этой категории.

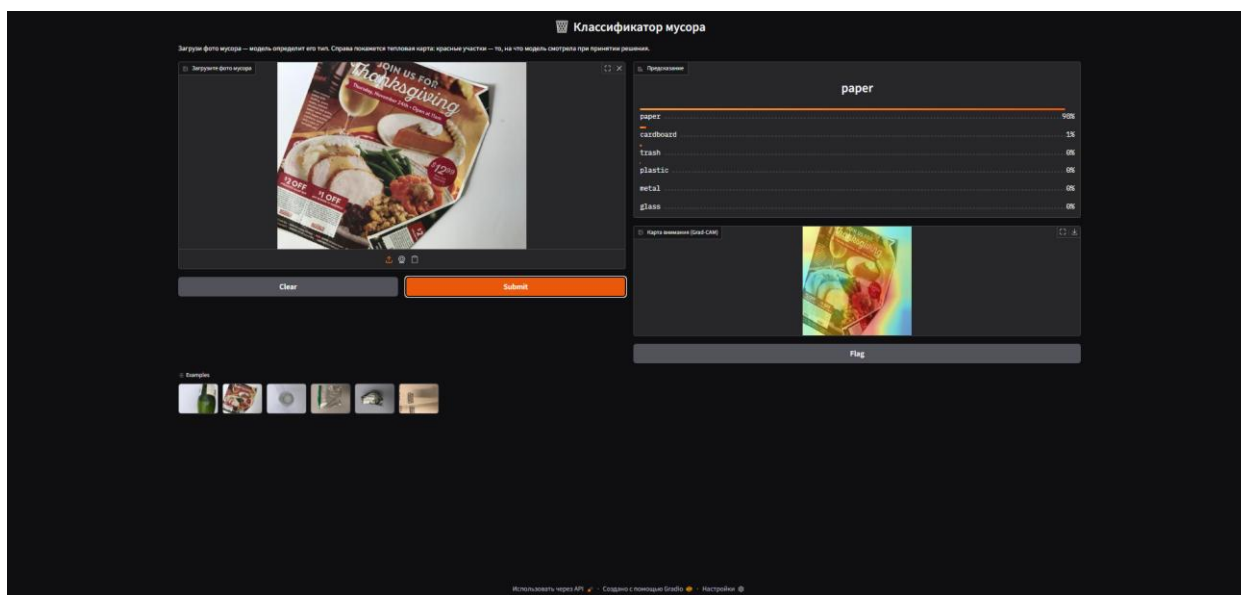


Рисунок 8 – Пример результата классификации (бумага)

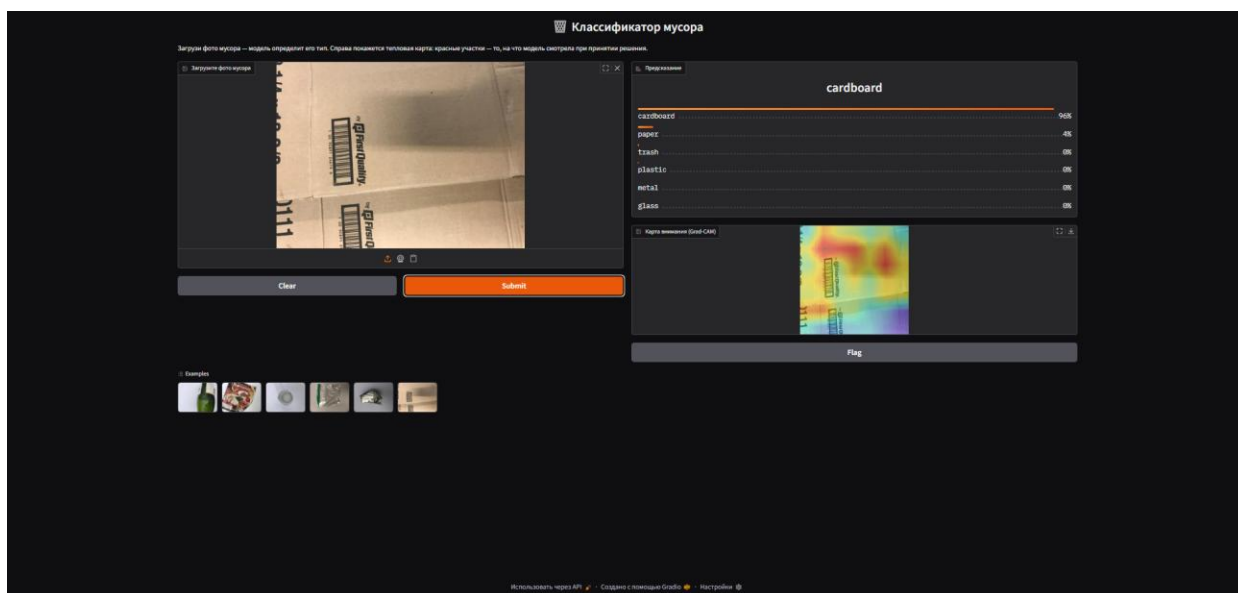


Рисунок 9 - Пример результата классификации (картон)

9.3 Анализ производительности

Скорость инференса модели составляет порядка 2–3 секунд на одно изображение на GPU T4, что вполне достаточно для интерактивного использования. На CPU инференс занимает порядка 10–20 секунд, что также приемлемо для веб-приложения. Расчёт Grad-CAM увеличивает время обработки примерно вдвое, но остаётся в пределах нескольких секунд, что не нарушает интерактивность интерфейса.

10 Заключение и направление развития

В рамках данной работы была разработана и обучена нейросетевая модель для автоматической классификации мусора по фотографии. Реализация выполнена на языке Python с использованием библиотеки TensorFlow и облачной среды Google Colab. Применение методики transfer learning на базе модели MobileNetV2 позволило получить рабочее решение в условиях ограниченного объёма данных и вычислительных ресурсов.

Помимо самой модели, был реализован алгоритм Grad-CAM для визуализации внимания модели — важный элемент объяснимого ИИ, повышающий доверие пользователя к системе и помогающий в анализе её ошибок. Создан удобный веб-интерфейс на базе Gradio с готовыми примерами и двумя видами вывода (классификация и тепловая карта), доступный с любого устройства через публичную ссылку.

Основные результаты работы можно сформулировать следующим образом. Во-первых, изучены и применены на практике современные методы компьютерного зрения: свёрточные нейронные сети, transfer learning, объяснимый ИИ. Во-вторых, успешно решена прикладная задача классификации мусора с приемлемым качеством. В-третьих, создан законченный программный продукт — от обработки данных до интерактивного интерфейса.

10.1 Количественные результаты

Обученная модель достигла общей точности (accuracy) 76,5 % на валидационной выборке из 503 изображений. Усреднённое значение F1-меры составило 0,720 (macro average) и 0,762 (weighted average). Детальные показатели по каждому классу приведены в таблице 1.

Таблица 1. Метрики качества классификации по классам

Класс	Precision	Recall	F1-score	Кол-во фото
Картон	0,901	0,800	0,848	80
Стекло	0,672	0,780	0,722	100
Металл	0,758	0,841	0,798	82
Бумага	0,828	0,898	0,862	118
Пластик	0,763	0,604	0,674	96
Прочий мусор	0,476	0,370	0,417	27
Macro avg	0,733	0,716	0,720	503
Weighted avg	0,766	0,765	0,762	503

Наилучшие показатели получены на классах «бумага» ($F1 = 0,862$) и «картон» ($F1 = 0,848$). Это объясняется наибольшим количеством обучающих изображений в этих категориях и характерными визуальными признаками — текстурой целлюлозных материалов, рёбрами картонной упаковки. Наихудший результат показал класс «прочий мусор» ($F1 = 0,417$): сказывается малый объём обучающей выборки (137 изображений против 400–600 у остальных) и высокое внутреннее разнообразие категории, в которую попадают объекты различной природы.

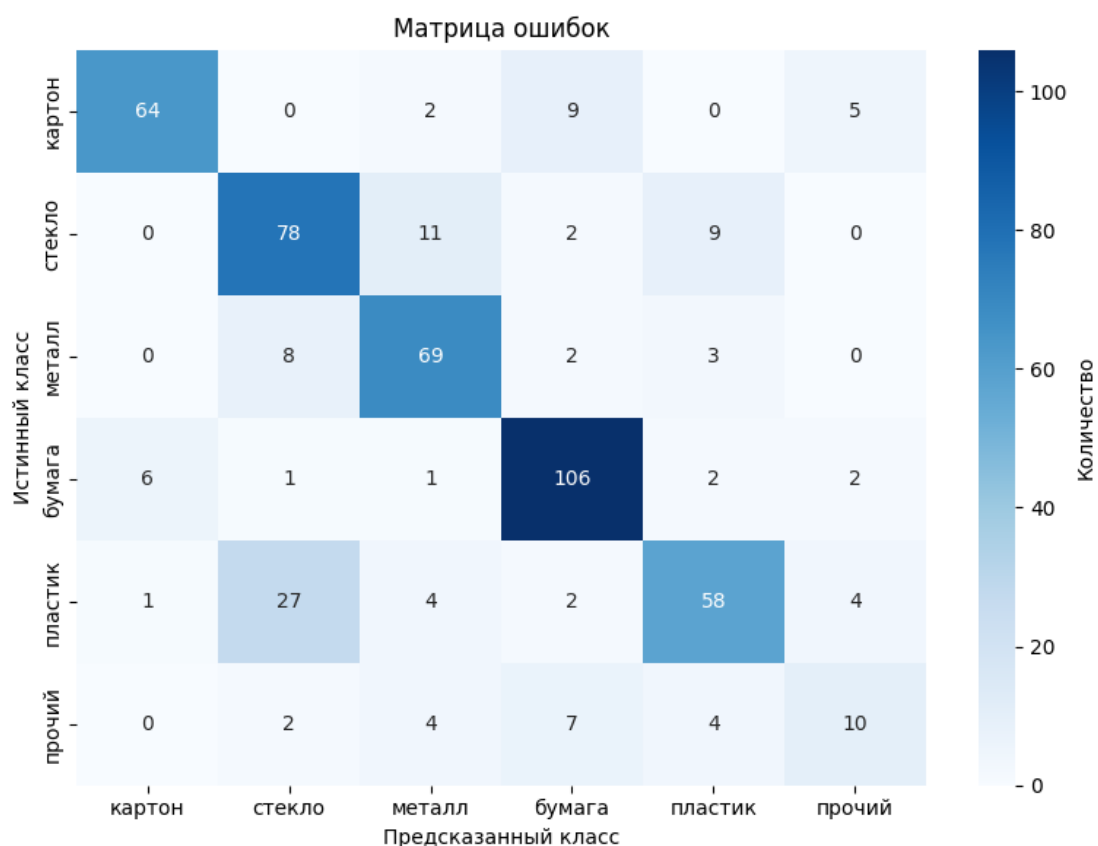


Рисунок 10 – Матрица ошибок классификатора

Детальное распределение ошибок классификации представлено в виде матрицы ошибок на рисунке 10. По её строкам отложены истинные классы, по столбцам — предсказанные.

Анализ матрицы ошибок позволяет выявить наиболее проблемные пары классов. Основная путаница происходит между классами «пластик» и «стекло»: 27 из 96 пластиковых объектов модель ошибочно отнесла к стеклу. Это объясняется визуальной схожестью материалов — оба часто прозрачные, имеют похожую форму ёмкостей и бликующие поверхности. Также наблюдается путаница «стекло» → «металл» (11 случаев) и «картон» → «бумага» (9 случаев), что обусловлено схожестью текстур и цветовых характеристик. Класс «прочий мусор» равномерно путается со всеми остальными, что подтверждает гипотезу о недостаточном объёме обучающих данных и слишком широком определении этой категории.

Сравнение метрик precision, recall и F1-score по классам представлено на рисунке 11. Видно, что для большинства классов precision и recall сбалансированы. Единственное заметное расхождение — у класса «пластик» (precision 0,763, recall 0,604), что говорит о тенденции модели «недопредсказывать» этот класс, ошибочно относя его объекты к другим категориям.

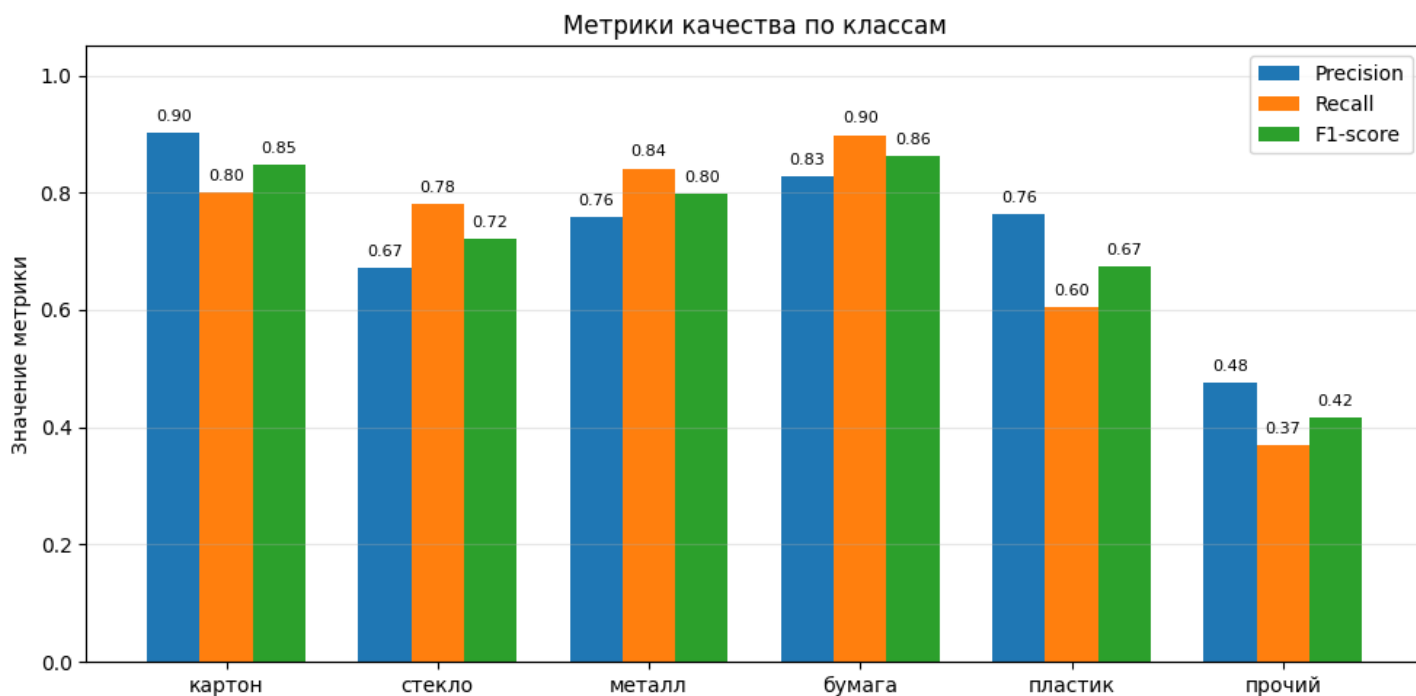


Рисунок 11 – Сравнение метрик precision, recall и F1-score по классам

10.2 Возможные направления дальнейшего развития:

1. Расширение датасета — добавление новых изображений, особенно для недостаточно представленных классов (trash), а также сбор фотографий в реальных условиях съёмки (мусорные баки, свалки, конвейерные ленты), что повысит устойчивость модели к разнообразию реальных условий.
2. Fine-tuning базовой модели — разморозка верхних слоёв MobileNetV2 и их дообучение с малой скоростью обучения. Это

потенциально повысит точность за счёт лучшей адаптации признаков к специфике задачи.

3. Балансировка классов — применение методов борьбы с дисбалансом: взвешивания классов в функции потерь, oversampling редких классов, генерация синтетических изображений через GAN.
4. Переход от классификации к детекции объектов — использование архитектур семейства YOLO или Faster R-CNN, что позволит работать с фотографиями, содержащими несколько объектов одновременно, и определять не только тип, но и местоположение каждого из них.

Проделанная работа демонстрирует, что современные методы машинного обучения позволяют создавать прикладные решения в области экологии и переработки даже силами одного разработчика и без специализированного оборудования. Сочетание точной модели с прозрачностью её решений через объяснимый ИИ делает такие системы пригодными не только для автоматических конвейерных линий, но и для пользовательских приложений, где доверие конечного потребителя является ключевым фактором успеха.

11 Листинг кода

```
!pip install gradio kaggle -q

""""# Новый раздел""""

from google.colab import userdata
import os

os.environ['KAGGLE_TOKEN'] = userdata.get('KAGGLE_TOKEN')

# Скачиваем датасет
!kaggle datasets download -d asdasdasdasdas/garbage-classification
!unzip -q garbage-classification.zip -d data

data_dir = '/content/data/Garbage classification/Garbage classification'
classes = os.listdir(data_dir)
print("Классы:", classes)

for cls in classes:
    count = len(os.listdir(os.path.join(data_dir, cls)))
    print(f" {cls}: {count} фото")

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

data_dir = '/content/data/Garbage classification/Garbage classification'

# Аугментация – искусственно увеличиваем разнообразие данных
# поворачиваем, отражаем, зумируем фото чтобы модель лучше обобщала
datagen = ImageDataGenerator(
    rescale=1./255,          # нормализация пикселей 0-255 → 0-1
    validation_split=0.2,    # 20% данных – на проверку
    rotation_range=20,
    horizontal_flip=True,
    zoom_range=0.2
)

train_data = datagen.flow_from_directory(
    data_dir,
    target_size=(224, 224),
    batch_size=32,
    subset='training'
)

val_data = datagen.flow_from_directory(
    data_dir,
    target_size=(224, 224),
```

```

        batch_size=32,
        subset='validation'
    )

    print("Классы:", train_data.class_indices)

    from tensorflow.keras.applications import MobileNetV2
    from tensorflow.keras import layers, models

    # Загружаем MobileNetV2 без последнего слоя
    base_model = MobileNetV2(
        input_shape=(224, 224, 3),
        include_top=False, # убираем оригинальный классификатор
        weights='imagenet' # веса с обучения на миллионах картинок
    )
    base_model.trainable = False # замораживаем — не трогаем чужие веса

    # Добавляем свой классификатор под 6 классов
    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(6, activation='softmax') # 6 классов мусора
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.summary()

    history = model.fit(
        train_data,
        epochs=10,
        validation_data=val_data,
        callbacks=[
            tf.keras.callbacks.EarlyStopping(
                patience=3, # остановить если 3 эпохи нет улучшений
                restore_best_weights=True
            )
        ]
    )

    import matplotlib.pyplot as plt

```

```

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Обучение')
plt.plot(history.history['val_accuracy'], label='Валидация')
plt.title('Точность')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Обучение')
plt.plot(history.history['val_loss'], label='Валидация')
plt.title('Ошибка')
plt.legend()

plt.show()

import gradio as gr
import numpy as np
import cv2
import tensorflow as tf
import os
import random

class_names = {v: k for k, v in train_data.class_indices.items()}

def make_gradcam_heatmap(img_array, model, last_conv_layer_name="Conv_1"):
    base_model = model.layers[0] # MobileNetV2 (Functional, у него output
    есть)
    last_conv_layer = base_model.get_layer(last_conv_layer_name)

    # Модель: вход → (карта признаков, выход base_model)
    feature_model = tf.keras.Model(
        inputs=base_model.input,
        outputs=[last_conv_layer.output, base_model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, base_output = feature_model(img_array)

        # Прогоняем через остальные слои (pooling, dense, dropout, softmax)
        x = base_output
        for layer in model.layers[1:]:
            x = layer(x)
        predictions = x

    pred_index = tf.argmax(predictions[0])
    class_channel = predictions[:, pred_index]

```

```

# Градиенты предсказанного класса по карте признаков
grads = tape.gradient(class_channel, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

# Взвешиваем карты по важности
conv_outputs = conv_outputs[0]
heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
heatmap = tf.squeeze(heatmap)

# Нормализуем в [0, 1]
heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-
10)
return heatmap.numpy()

def overlay_heatmap(img, heatmap, alpha=0.4):
    """Накладываем тепловую карту на оригинал."""
    heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap_colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)
    overlay = cv2.addWeighted(img.astype(np.uint8), 1 - alpha,
heatmap_colored, alpha, 0)
    return overlay

def predict(img):
    if img is None:
        return None, None

    img_resized = tf.image.resize(img, (224, 224)).numpy()
    img_for_model = np.expand_dims(img_resized / 255.0,
0).astype(np.float32)

    # Предсказание
    predictions = model.predict(img_for_model, verbose=0)[0]
    probs = {class_names[i]: float(predictions[i]) for i in
range(len(class_names))}

    # Тепловая карта
    heatmap = make_gradcam_heatmap(img_for_model, model)
    overlay = overlay_heatmap(img_resized.astype(np.uint8), heatmap)

    return probs, overlay

example_paths = []
for cls in os.listdir(data_dir):
    cls_dir = os.path.join(data_dir, cls)
    files = os.listdir(cls_dir)

```

```
example_paths.append(os.path.join(cls_dir, random.choice(files)))
```

```
# === Интерфейс ===
app = gr.Interface(
    fn=predict,
    inputs=gr.Image(label="Загрузите фото мусора"),
    outputs=[
        gr.Label(num_top_classes=6, label="Предсказание"),
        gr.Image(label="Карта внимания (Grad-CAM)")
    ],
    title="🗑️ Классификатор мусора",
    description=(
        "Загрузи фото мусора – модель определит его тип. "
        "Справа покажется тепловая карта: красные участки – то, "
        "на что модель смотрела при принятии решения."
    ),
    examples=example_paths
)

app.launch(share=True, debug=True)
```